Step 6 — Backend Folder Structure (FastAPI Project)

1. Introduction

In a professional backend application, code should be organized in a structured way rather than writing everything inside one file. A proper folder structure makes the project easier to understand, maintain, debug, and scale in the future.

For our AI-powered Data Visualization Application, we will use a professional backend architecture using Python and FastAPI.

Backend Folder Structure

```
backend
│
├── app
│   │
│   ├── main.py
│   │
│   ├── database.py
│   │
│   ├── models
│   │   └── user.py
│   │
│   ├── routes
│   │   └── auth.py
│   │
│   ├── schemas
│   │   └── user_schema.py
│   │
│   ├── services
│   │   └── auth_service.py
│   │
│   ├── utils
│   │   └── jwt_handler.py
│   │
│   └── config.py
│
└── requirements.txt
```

---

2. Why This Structure?

This structure follows a **professional backend architecture** used in real-world software development.

Benefits of Using This Structure

- Makes the project clean and organized.
- Helps developers understand the code easily.
- Reduces confusion in large projects.
- Makes debugging easier.
- Allows multiple developers to work together.
- Follows industry-standard development practices.
- Easy to maintain and scale in the future.

Instead of writing all backend code in one file, different responsibilities are separated into different folders.

This concept is called **Separation of Concerns (SoC)**.

---

3. What is Separation of Concerns?

Separation of Concerns means dividing the application into different sections, where each section has a specific responsibility.

For example:

- Database-related code goes into `models`.
- API endpoint code goes into `routes`.
- Business logic goes into `services`.
- Validation code goes into `schemas`.

This improves readability and reduces code complexity.

---

4. Explanation of Each File and Folder

4.1 `main.py`

Purpose

This is the **main entry point** of the FastAPI application.

It starts the backend server and connects all routes.

Responsibilities

- Create FastAPI application.
- Register API routes.
- Start backend services.
- Handle main server configuration.

Example Use

When we run:

```
uvicorn app.main:app --reload
```

FastAPI starts execution from `main.py`.

4.2 `database.py`

Purpose

This file handles **database connection setup**.

Responsibilities

- Connect backend to database.
- Configure database URL.
- Manage database sessions.

Example Use

If using MongoDB or PostgreSQL, connection logic is written here.

Without this file, backend cannot communicate with database.

4.3 `models/`

Purpose

This folder stores **database models (tables/collections)**.

Meaning

A model represents how data is stored in the database.

Example

`user.py`

Stores user-related database structure such as:

- Name
- Email
- Password
- User role
- Account details

Responsibilities

- Define database tables.
- Store database structure.
- Manage user/entity models.

In short:

models = database tables

4.4 `routes/`

Purpose

This folder contains **API endpoints**.

Meaning

Routes decide what happens when frontend sends requests to backend.

Example

`auth.py`

Handles authentication APIs:

- Signup API
- Login API
- Logout API
- Forgot password API

Responsibilities

- Define API endpoints.
- Handle HTTP requests.
- Connect frontend and backend.

Example route:

```
@app.get("/")
def home():
    return {"message": "Backend Running"}
```

In short:

routes = API endpoints

4.5 `schemas/`

Purpose

This folder is used for **data validation**.

Meaning

Schemas validate user input before storing data in database.

Example

`user_schema.py`

Checks:

- Email format is correct or not.
- Password length is valid.
- Required fields exist.

Responsibilities

- Input validation.
- Request validation.
- Response formatting.

Example:

If user enters invalid email:

```
abc123
```

Schema prevents incorrect data from entering database.

In short:

schemas = validation layer

4.6 `services/`

Purpose

This folder contains **business logic**.

Meaning

Business logic means the actual working process of the application.

Example

`auth_service.py`

Handles:

- Password hashing
- User authentication
- Login verification
- JWT token generation

Responsibilities

- Main application logic.
- Data processing.
- Authentication logic.

Example:

Instead of writing login logic in route file, we write it in service file.

This keeps code clean.

In short:

services = business logic

4.7 `utils/`

Purpose

This folder stores **reusable helper functions**.

Meaning

Utility functions are reusable functions used multiple times.

Example

`jwt_handler.py`

Handles:

- Create JWT token
- Verify JWT token
- Decode token

Responsibilities

- Reusable helper functions.
- Common utility logic.
- Security-related helpers.

In short:

utils = reusable functions

4.8 `config.py`

Purpose

Stores **application settings and configurations**.

Responsibilities

- Environment variables.
- Secret keys.
- Database URL.
- API configurations.

Example:

```
SECRET_KEY = "your_secret_key"
DATABASE_URL = "mongodb://localhost"
```

This keeps sensitive information separate.

4.9 `requirements.txt`

Purpose

Stores all Python packages used in the backend.

Responsibilities

- Package management.
- Dependency installation.

Example packages:

- FastAPI
- Uvicorn
- Pydantic
- JWT

- Database libraries

Install all packages using:

```
pip install -r requirements.txt
```

---

5. Summary

Folder/File	Purpose
`main.py`	Starts FastAPI server
`database.py`	Database connection
`models/`	Database tables
`routes/`	API endpoints
`schemas/`	Validation
`services/`	Business logic
`utils/`	Reusable functions
`config.py`	Configuration settings
`requirements.txt`	Python packages

Simple Formula to Remember

```
models → Database
routes → APIs
schemas → Validation
services → Logic
utils → Reusable functions
```